11

Laboratory 11

11. Camera calibration and stereovision

A camera is a device that converts the 3D world into a 2D images. This relationship can be described by the following equation:

$$x = PX \tag{11.1}$$

where: x denotes a 2-D image point, P denotes the camera matrix and X denotes a 3-D world point.

Linear or non-linear algorithms are used to estimate intrinsic and extrinsic parameters utilising known points in real-time and their projections in the picture plane.

Camera calibration is designed to eliminate distortions introduced by the optical system. These distortions include: radial distortions (barrel, pincushion, mustache – related to the lens) and tangential distortions (related to the uneven installation of the lens and the matrix – Figures 11.1 and 11.2.

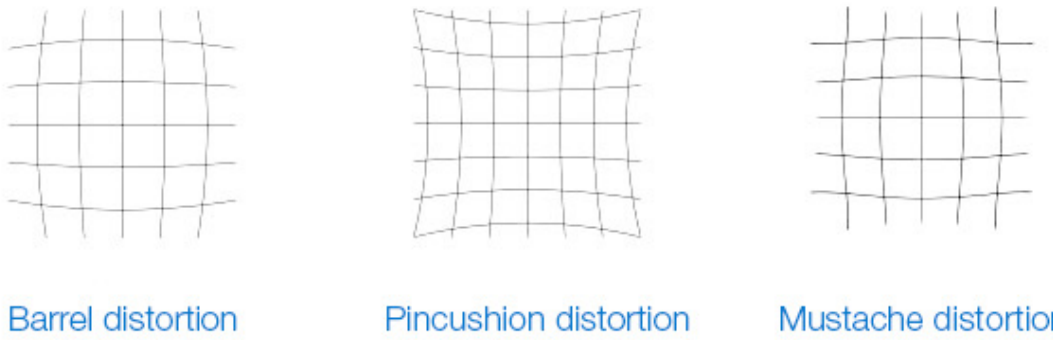

Figure 11.1: Types of distortion: (a) barrel, (b) pincushion (c) mustache. Source: https://www.panoramic-photo-guide.com/optical-distortions-panoramic-photography.html.

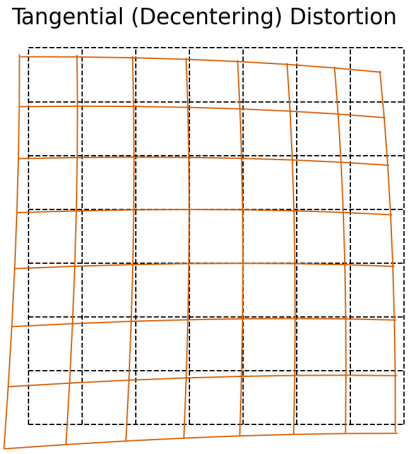

Figure 11.2: Tangential distortion. Source: https://www.tangramvision.com/blog/camera-modeling-exploring-distortion-and-distortion-models-part-i.

Distortion correction requires the calculation of some coefficients. This is possible in the calibration procedure, where an image of a pattern with known parameters (mutual distances between points and their actual sizes) is recorded at different positions relative to the camera. The most commonly used pattern is a chessboard, on which corners can be easily detected, even with sub-pixel accuracy (as in the analysed image). Practice indicates that there should be at least 10 images.

The parameters determined in the camera calibration procedure are divided into two groups:

- Intrinsic or Internal parameters – they allow mapping between pixel coordinates and camera coordinates in the image frame, e.g. optical centre, focal length, and radial distortion coefficients of the lens.
- Extrinsic or External parameters – they describe the orientation and location of the camera. This refers to the rotation and translation of the camera with respect to some world coordinate system.

Figure 11.3 shows the equation of the camera model, which contains the homogeneous coordinates and the projection matrix. The calibration matrix is the product of matrices containing internal and external camera parameters. Linear or non-linear algorithms are used to estimate the internal and external parameters. Knowledge of the real parameters (distance between points, in centimetres) and their projections on a plane (in pixels) is used.

There are two camera models:

- *pinhole camera model* – basic camera model without lens, see OpenCV documentation for details: Pinhole model,
- *fisheye camera model* – primarily used in situations where distortion is extreme, models well for wide angle cameras, see OpenCV documentation for details: Fisheye model.

Performing the correct distortion correction – camera calibration – is essential if you want to measure the size of objects, e.g. in centimetres, measure distance or determine camera displacement.

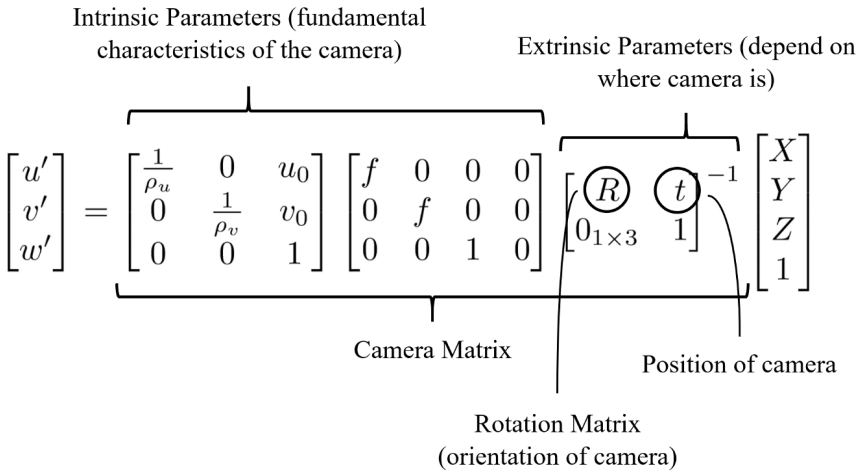

Figure 11.3: Camera model formula based on general equation 11.1. ρ_u, ρ_v is the image size x, y in pixels, u_0, v_0 is the principal point, f is the focal length. Source: https://towardsdatascience.com/understanding-transformations-in-computer-vision-b001f49a9e61.

11.1 Single camera calibration

The camera calibration procedure is divided into steps. Almost identical steps have to be followed for the calibration of a single camera or of stereovision cameras. A dataset containing image pairs from a stereovision camera will be used. To calibrate a single camera, a set from a single lens (left or right images respectively) should be selected. The proposed set was captured by a camera that needs to be modelled by a fisheye model.

In general, the camera calibration process can be divided into:

1. Selecting a proper calibration pattern. The most commonly used are: chessboard, ChArUco (combination of chessboard and ArUco markers), circle grid, asymmetric circle grid.
2. Taking pictures of the calibration board from different angles and distances. The number of correctly taken images should be greater than 10.
3. Preparing the parameters of the calibration board, e.g. measuring the size of the chessboard fields in centimetres.
4. Preparing the code and running the algorithm for calibration.

The steps of the algorithm for calibrating a single camera are described below. A previously prepared set of images (available on the UPeL platform) and functions from the OpenCV library were used for this.

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# termination criteria
criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)
calibration_flags = cv2.fisheye.CALIB_RECOMPUTE_EXTRINSIC+cv2.fisheye.
    CALIB_FIX_SKEW

# inner size of chessboard
width = 9
height = 6
square_size = 0.025  # 0.025 meters

# prepare object points, like (0,0,0), (1,0,0), (2,0,0) ....,(8,6,0)
```

```
objp = np.zeros((height * width, 1, 3), np.float64)
objp[:, 0, :2] = np.mgrid[0:width, 0:height].T.reshape(-1, 2)

objp = objp * square_size  # Create real world coords. Use your metric.

# Arrays to store object points and image points from all the images.
objpoints = []  # 3d point in real world space
imgpoints = []  # 2d points in image plane.

img_width = 640
img_height = 480
image_size = (img_width,img_height)

path = ""
image_dir = path + "pairs/"

number_of_images = 50
for i~in range(1, number_of_images):
    # read image
    img = cv2.imread(image_dir + "left_%02d.png" % i)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

    # Find the chess board corners
    ret, corners = cv2.findChessboardCorners(gray, (width, height), cv2.
        CALIB_CB_ADAPTIVE_THRESH + cv2.CALIB_CB_FAST_CHECK + cv2.
        CALIB_CB_NORMALIZE_IMAGE)

    Y, X, channels  =  img.shape

    # skip images where the corners of the chessboard are too close to the edges of
         the image
    if (ret == True):
        minRx = corners[:,:,0].min()
        maxRx = corners[:,:,0].max()
        minRy = corners[:,:,1].min()
        maxRy = corners[:,:,1].max()

        border_threshold_x = X/12
        border_threshold_y = Y/12

        x_thresh_bad = False
        if (minRx < border_threshold_x):
            x_thresh_bad = True

        y_thresh_bad = False
        if (minRy < border_threshold_y):
            y_thresh_bad = True

        if (y_thresh_bad==True) or (x_thresh_bad==True):
            continue

    # If found, add object points, image points (after refining them)
    if ret == True:
        objpoints.append(objp)

        # improving the location of points (sub-pixel)
        corners2 = cv2.cornerSubPix(gray, corners, (3, 3), (-1, -1), criteria)

        imgpoints.append(corners2)

        # Draw and display the corners
        # Show the image to see if pattern is found ! imshow function.
        cv2.drawChessboardCorners(img, (width, height), corners2, ret)
        cv2.imshow("Corners", img)
        cv2.waitKey(5)
    else:
        print("Chessboard couldn't detected. Image pair: ", i)
        continue
```

With the coordinates of the points calculated, you can proceed to calibration:

```
N_OK = len(objpoints)
K = np.zeros((3, 3))
D = np.zeros((4, 1))
rvecs = [np.zeros((1, 1, 3), dtype=np.float64) for i~in range(N_OK)]
tvecs = [np.zeros((1, 1, 3), dtype=np.float64) for i~in range(N_OK)]

ret, K, D, _, _ = \
    cv2.fisheye.calibrate(
        objpoints,
        imgpoints,
        image_size,
        K,
        D,
        rvecs,
        tvecs,
        calibration_flags,
        (cv2.TERM_CRITERIA_EPS+cv2.TERM_CRITERIA_MAX_ITER, 30, 1e-6)
    )
# Let's rectify our results
map1, map2 = cv2.fisheye.initUndistortRectifyMap(K, D, np.eye(3), K, image_size,
    cv2.CV_16SC2)
```

R Description of the parameters obtained during the calibration process:

- `ret` – the value of the mean squared errors of the back projection (describes the quality of the matching),
- `K` – matrix of internal camera parameters in the form:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \tag{11.2}$$

  where: (f_x, f_y) – focal length, (c_x, c_y) – principal points,
- `D` – distortion coefficients,
- `rvecs` – rotation vectors,
- `tvecs` – translation vectors (together with the rotation vectors allow to transform the position of the calibration standard from the model coordinates to the real coordinates).

Having obtained all the necessary parameters in the calibration process, it is now necessary to select any image from the given set and remove the distortion. To do this, the function `cv2.remap()` will be used:

```
undistorted_image = cv2.remap(image, map1, map2, interpolation=cv2.INTER_LINEAR,
    borderMode=cv2.BORDER_CONSTANT)
```

Exercise 11.1 Using the above description to calibrate a single camera, complete the task.

- Determine the camera parameters and display them.
- Check the correction for one of the images in the set (straight line test).
- Perform distortion correction for the entire set of images. Display and compare the image before and after calibration.

■

11.2 Stereo camera calibration

In the greatest simplification, the calibration of a camera system (for simplicity, two cameras) is meant to enable easy computation of depth maps. This problem can be understood in several ways:

- setting identical parameters for both cameras,
- elimination of distortions,
- mapping to a common coordinate system,
- rectification.

The last issue is particularly interesting.

In the general case, corresponding points in both images must lie on their respective epipolar lines. **Rectification** consists of applying appropriate (usually projective) transformations to both images so that corresponding epipolar lines become collinear and parallel to one of the image edges (most often the horizontal one). In practice, this means that for corresponding points their vertical coordinate is the same, and the difference (disparity) appears only in the horizontal direction. This greatly facilitates establishing correspondences (the depth map) – the search is then carried out only in one direction (along the image rows).

It is worth knowing that OpenCV also provides a function for rectifying uncalibrated images, `stereoRectifyUncalibrated`. We encourage you to running it.

Exercise 11.2 Carry out the calibration and rectification process for a stereo camera system. Display the images after distortion removal and overlay horizontal lines on them to confirm that the algorithm works correctly.

General procedure:

1. Use the code for single-camera calibration as a starting point.
2. Introduce modifications to take into account the analysis of a two-camera system.
3. Determine the intrinsic parameters (and distortion) separately for the left and right images.
4. Perform stereo calibration using the obtained camera parameters (you can use the code in Listing 11.1).
5. Perform rectification and display the rectified (calibrated) images.
6. Merge the images and draw horizontal lines on them to check whether corresponding points lie in the same rows.

■

Listing 11.1: Stereo calibration

```
imgpointsLeft = np.asarray(imgpointsLeft, dtype=np.float64)
imgpointsRight = np.asarray(imgpointsRight, dtype=np.float64)

(RMS, _, _, _, _, rotationMatrix, translationVector) = cv2.fisheye.stereoCalibrate(
        objpoints, imgpointsLeft, imgpointsRight,
        K_left, D_left,
        K_right, D_right,
        image_size, None, None,
        cv2.CALIB_FIX_INTRINSIC,
        (cv2.TERM_CRITERIA_EPS+cv2.TERM_CRITERIA_MAX_ITER, 30, 0.01))

R2 = np.zeros([3,3])
P1 = np.zeros([3,4])
P2 = np.zeros([3,4])
Q = np.zeros([4,4])

# Rectify calibration results
(leftRectification, rightRectification, leftProjection, rightProjection,
    dispartityToDepthMap) = cv2.fisheye.stereoRectify(
```

```
        K_left, D_left,
        K_right, D_right,
        image_size,
        rotationMatrix, translationVector,
        0, R2, P1, P2, Q,
        cv2.CALIB_ZERO_DISPARITY, (0,0) , 0, 0)

map1_left, map2_left = cv2.fisheye.initUndistortRectifyMap(
        K_left, D_left, leftRectification,
        leftProjection, image_size, cv2.CV_16SC2)

map1_right, map2_right = cv2.fisheye.initUndistortRectifyMap(
        K_right, D_right, rightRectification,
        rightProjection, image_size, cv2.CV_16SC2)


dst_L = cv2.remap(img_l, map1_left, map2_left, cv2.INTER_LINEAR)
dst_R = cv2.remap(img_r, map1_right, map2_right, cv2.INTER_LINEAR)
```

R Description of the parameters used in the above methods:

- `imgpointsLeft, imgpointsRight` – list of detected chessboard corners for the left and right image, the values come from the function `cv2.cornerSubPix`,
- K_left, K_right – matrix of internal camera parameters for a set of image pairs, the values come from the function `cv2.fisheye.calibrate`,
- D_left, D_right – camera distortion coefficients, values are derived from the function `cv2.fisheye.calibrate`.

11.3 Stereo correspondence problem

Determining stereo correspondence itself (a disparity map and, consequently, a depth map) is relatively simple — at least in theory and for suitable images. The goal is to find by how much the pixel $I_L(x,y)$ is shifted in the second image, i.e. which pixel $I_R(x+d,y)$ corresponds to it, where d denotes the disparity. To recall: we search only along horizontal lines, because we assume that the images have been rectified. It can also be noted that this task is in some sense similar to optical flow. There, we had two consecutive frames from a sequence recorded by a single camera, whereas here we have two images from two cameras at the same time. It is also worth knowing that depth can be recovered from a sequence from a single camera, provided the camera is moving — this is the *Structure from Motion* problem.

The OpenCV library provides two popular methods for computing a disparity map: block matching (*Block Matching*, `StereoBM`) and *Semi-Global Block Matching* (`StereoSGBM`). The operation of the first is quite intuitive; in the case of the second, (simplified) global optimisation is additionally applied, which usually improves the continuity of the map.

Exercise 11.3 Please, based on the documentation, determine the disparity map for one of the images from the *example* folder before and after the calibration process. Please choose the parameters of the individual functions empirically, following the constraints. Display the original image before and after calibration and the corresponding disparity maps (SGM and BM).

An example solution is presented in Figure 11.4. ■

R The disparity map should be normalised to the range $0-255$ before display. You can also apply a function to convert the disparity to a heatmap: `cv2.applyColorMap(map, cv2.COLORMAP_HOT)`.

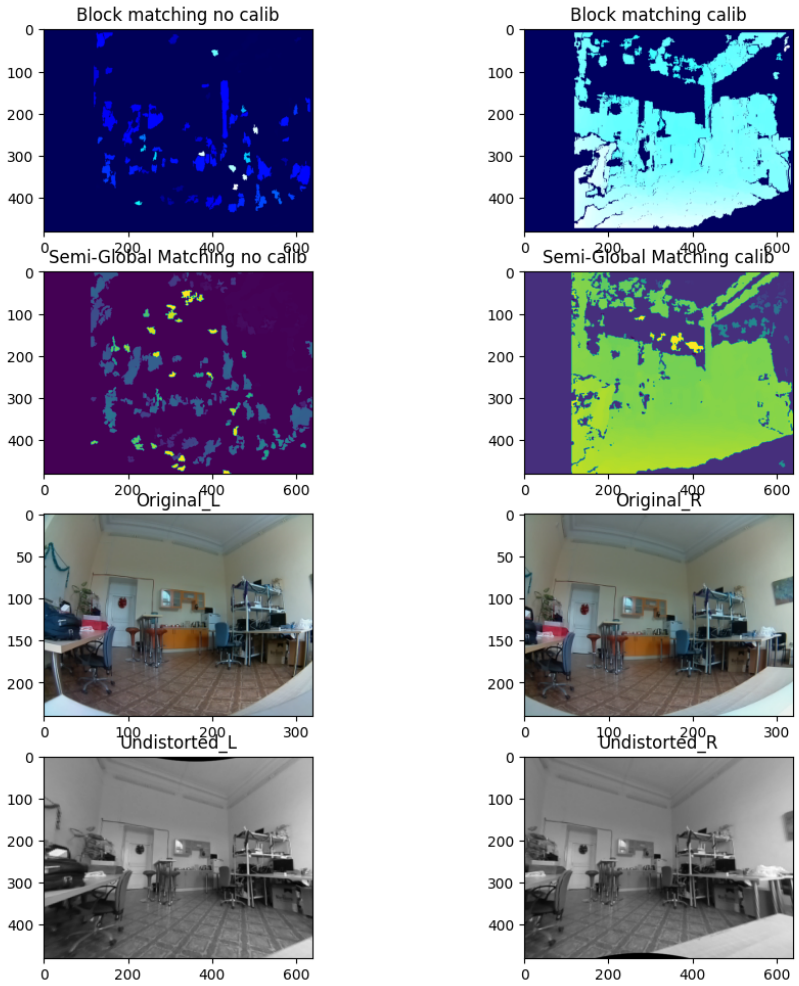

Figure 11.4: An example solution.

11.4 Using a neural network for the task of image depth analysis

As an exercise, we propose to analyse a deep convolutional neural network that allows to obtain depth maps from **single-view depth** method. In addition, the network is trained in an unsupervised manner (i.e. without a teacher – in this case without reference depth maps). Furthermore, the network implements camera position estimation. The details of the method used can be read in the paper ZHOU, Tinghui, et al., Unsupervised Learning of Depth and Ego-Motion from Video. We will use the network model provided by the authors. The code is available on the GitHub repository – SfmLearner-Pytorch.

Exercise 11.4 Non-obligatory assignment. Run the unsupervised SfmLearner neural network.

To run inference with the neural network, you need to:

- Set the path to the trained network model,
- Set the path to the test images,
- Set the path where the output images are to be saved,
- Set the `output-disp, output-depth` flags so that the network returns the disparity and depth images.

```
python3 run_inference.py --pretrained 'dispnet_model_best.pth.tar' --dataset-dir
    'test_image/' --output-dir '/' --output-disp --output-depth
```

Compare the results obtained with the DCNN method with the classic BM and SGM methods (visually). ■